COMP 532
Machine Learning and Bioinspired Optimisation

Lecture 8 Reinforcement Learning

UNIVERSITY OF LIVERPOOL

Meng Fang
University of Liverpool

Recap

- Reinforcement Learning
  - Value function

Overview

- Problem Solving from Nature
- Reinforcement Learning (RL)
  - Roots of RL
  - Key features of RL
  - Elements of RL
  - Multi-armed bandits
  - Solve the problem in Markov Decision Processes (MDPs)
  - Value function
  - Solve the Bellman Optimality Equation
- Deep Learning
- Multi-Agent Learning
- Swam Intelligence

Solve the Bellman Optimality Equation

The Three Ways to Solve Bellman Equation

We want to solve:

$$V = T^{\pi}V$$

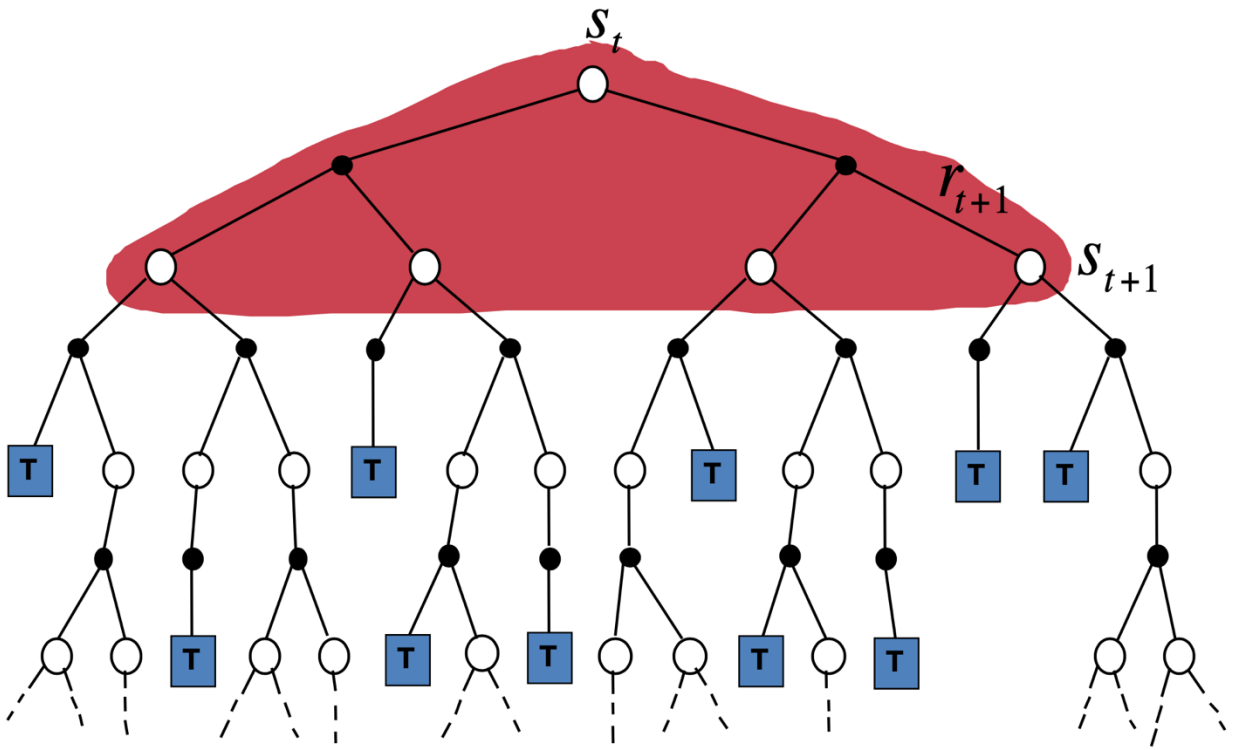
There are three approaches:

- Dynamic Programming
  → use full expectation operator
- Monte Carlo
  → replace expectation with sample average of full returns
- Temporal Difference
  → replace expectation with one-step bootstrap sample

Many RL methods can be understood as approximately solving the Bellman Optimality Equation.

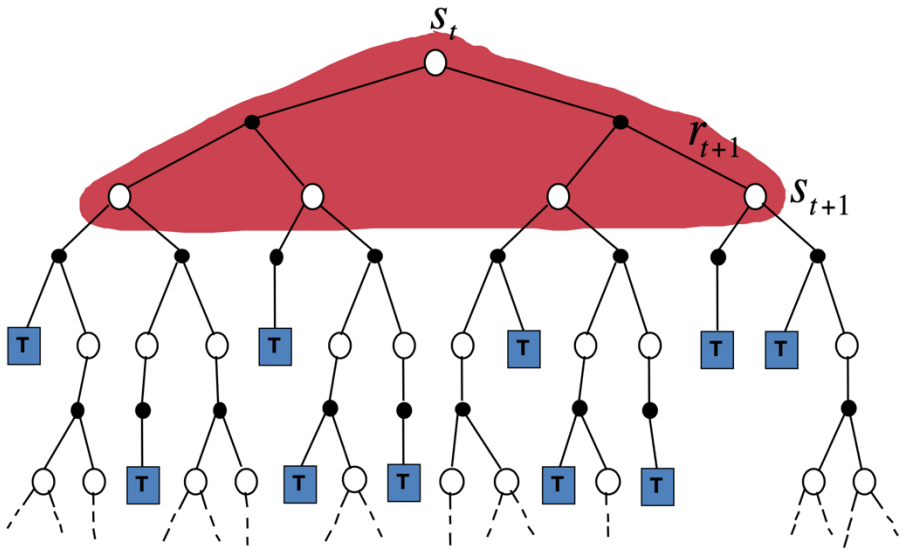
Dynamic Programming

- $V(s) \leftarrow \sum_a \pi(a \mid s) \sum_{s'} P^a_{ss'} \left[R^a_{ss'} + \gamma V(s') \right]$



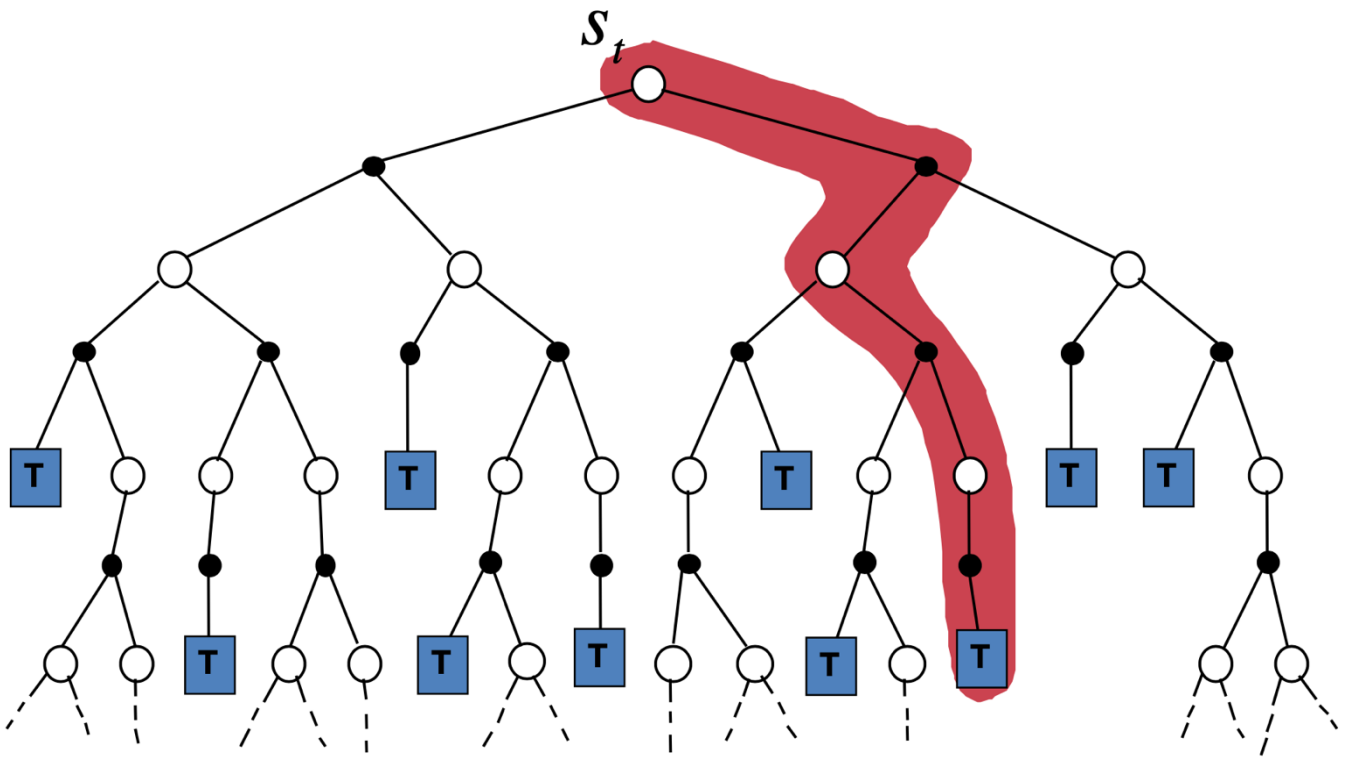
Dynamic Programming

- Finding optimal policy by explicitly solving Bellman:
  - $V(s) \leftarrow \sum_a \pi(a \mid s) \sum_{s'} P^a_{ss'} \left[R^a_{ss'} + \gamma V(s') \right]$
    - accurate knowledge of environment dynamics;
    - enough space and time to do the computation;
    - the Markov Property.
- Requires a full model and is computationally expensive for large state spaces.



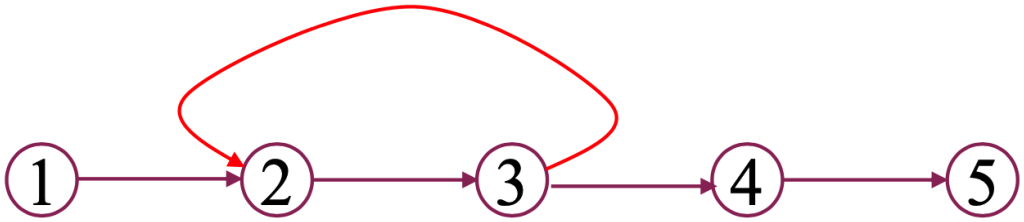
Monte Carlo method

- Monte Carlo methods learn from complete sample returns
  - Only defined for episodic tasks
- Monte Carlo methods learn directly from experience



Monte Carlo Policy Evaluation

- Goal: learn $V^{\pi}(s)$
- Given: some number of episodes under π which contain s
- Idea: Average returns observed after visits to s



- Every-Visit MC: average returns for every time s is visited in an episode
- First-visit MC: average returns only for first time s is visited in an episode
- Both converge asymptotically

First-visit Monte Carlo policy evaluation

- Average

Initialize:

$\pi \leftarrow$ policy to be evaluated

$V \leftarrow$ an arbitrary state-value function

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Repeat forever:

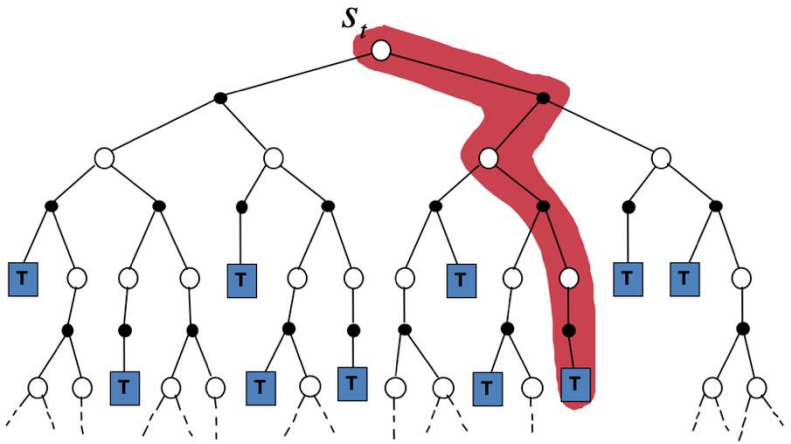
(a) Generate an episode using π

(b) For each state s appearing in the episode:

$R \leftarrow$ return following the first occurrence of s

Append R to $Returns(s)$

$V(s) \leftarrow$ average$(Returns(s))$



Backup diagram for Monte Carlo

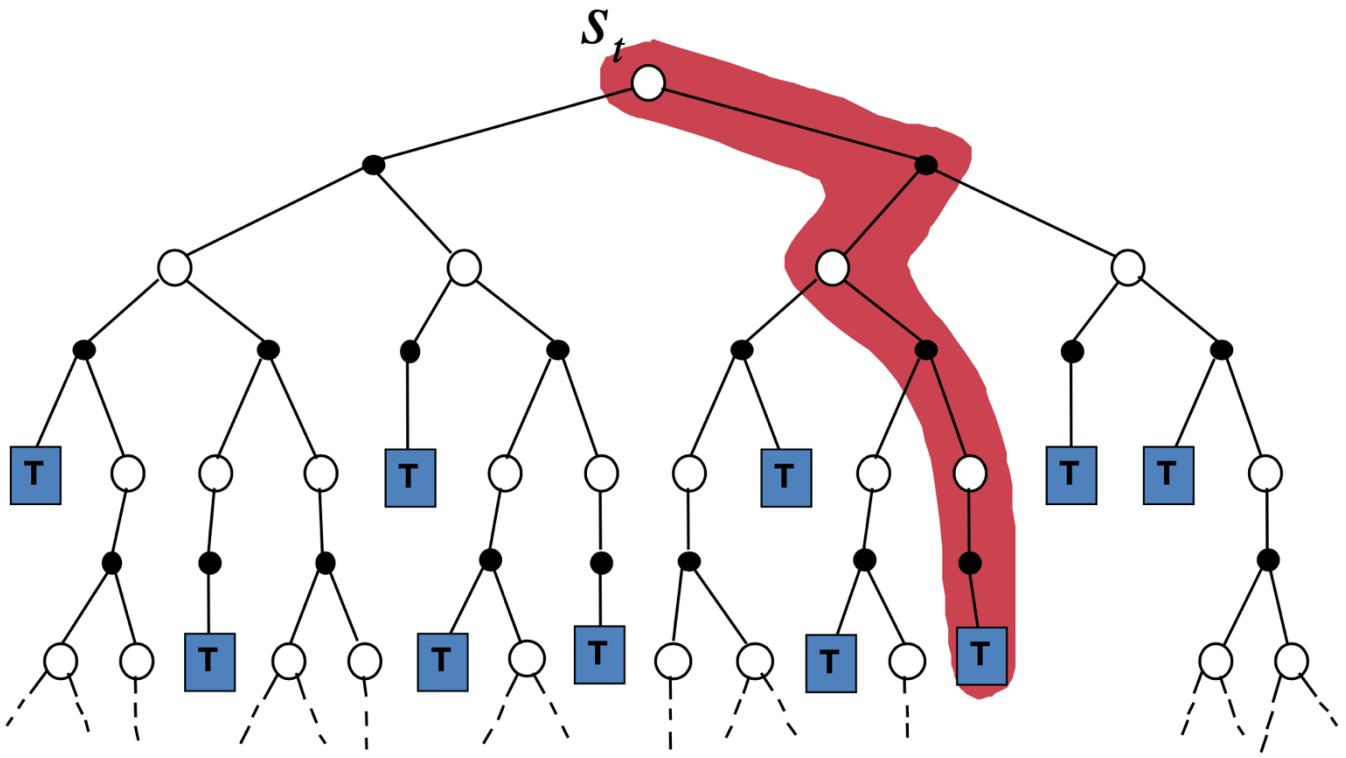
- Entire episode included
- Only one choice at each state (unlike DP)
- MC does not bootstrap
- Time required to estimate one state does not depend on the total number of states

Note: Bootstrap means using one or more estimated values in the update step for the same kind of estimated value.

Terminal state

Incremental Monte Carlo Update

- $V(s_t) \leftarrow V(s_t) + \alpha[R_{t+1} - V(s_t)]$ where R_t is the actual return following state s_t.

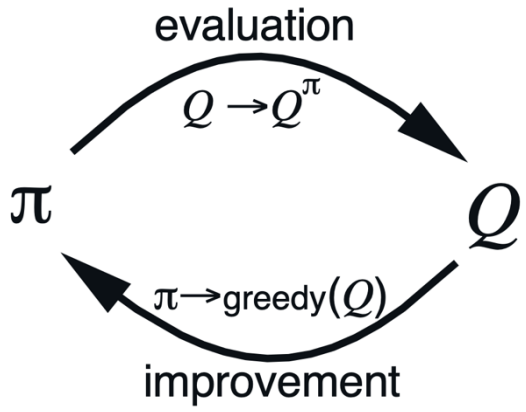


Monte Carlo Estimation of Action Values (Q)

- Monte Carlo is most useful when a model is not available
  - We want to learn Q*
- $Q^{\pi}(s, a)$ - average return starting from state s and action a following π
- Also converges asymptotically if every state-action pair is visited
- Exploring starts: Every state-action pair has a non-zero probability of being the starting pair

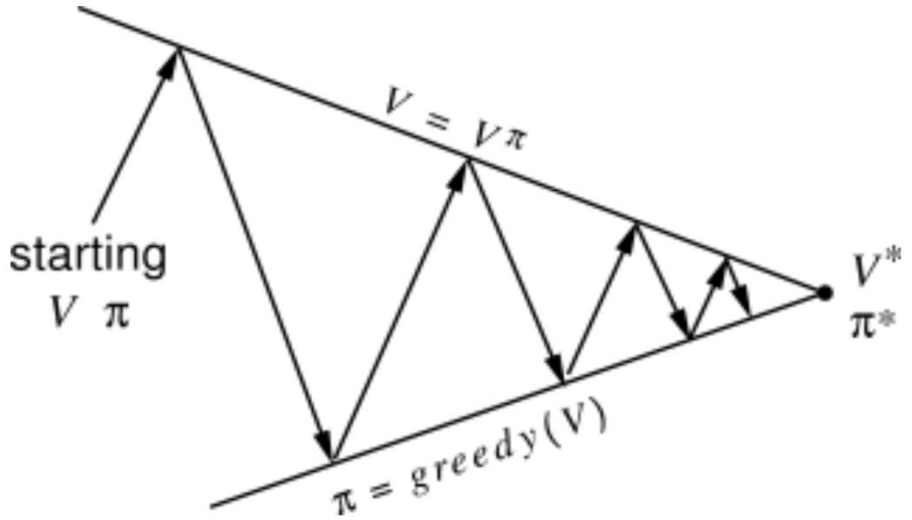
Monte Carlo Control

- MC policy iteration: Policy evaluation using MC methods followed by policy improvement
- Policy improvement step: greedy with respect to value (or action-value) function



Convergence of Monte Carlo Control

- Monte Carlo is most useful when a model is not available
- Interaction between policy evaluation and improvement
- iterative improvement until convergence



Convergence of Monte Carlo Control

- Greedified policy meets the conditions for policy improvement:

$$\begin{aligned} Q^{\pi_k}(s, \pi_{k+1}(s)) &= Q^{\pi_k}(s, \arg\max_a Q^{\pi_k}(s, a)) \\ &= \max_a Q^{\pi_k}(s, a) \\ &\geq Q^{\pi_k}(s, \pi_k(s)) \\ &\geq V^{\pi_k}(s). \end{aligned}$$

- And thus must be $\geq \pi_k$ by the policy improvement theorem
- This assumes exploring starts and infinite number of episodes for MC policy evaluation.

Monte Carlo is important in practice

- Absolutely
- When there are just a few possibilities to value, out of a large state space, Monte Carlo is a big win
- Backgammon, Go, ...

Temporal Difference method

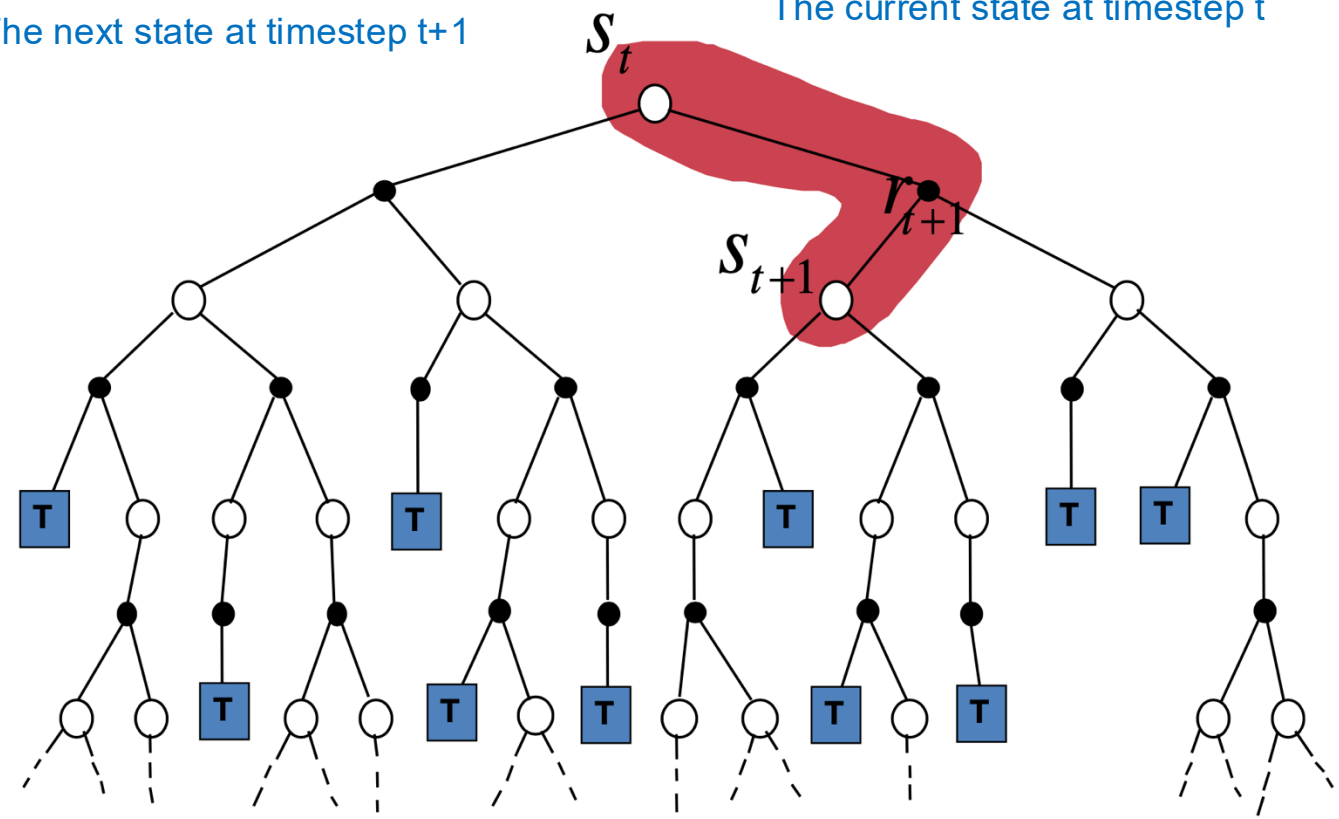
- The update rule of dynamic programming requires to know the dynamics of the environment.
- Typical is to use temporal difference methods to overcome this problem, like Q-learning.
- Look at the difference between the current estimate of the value of a state and the discounted value of the next state and the reward received.

TD(0) Update Rule

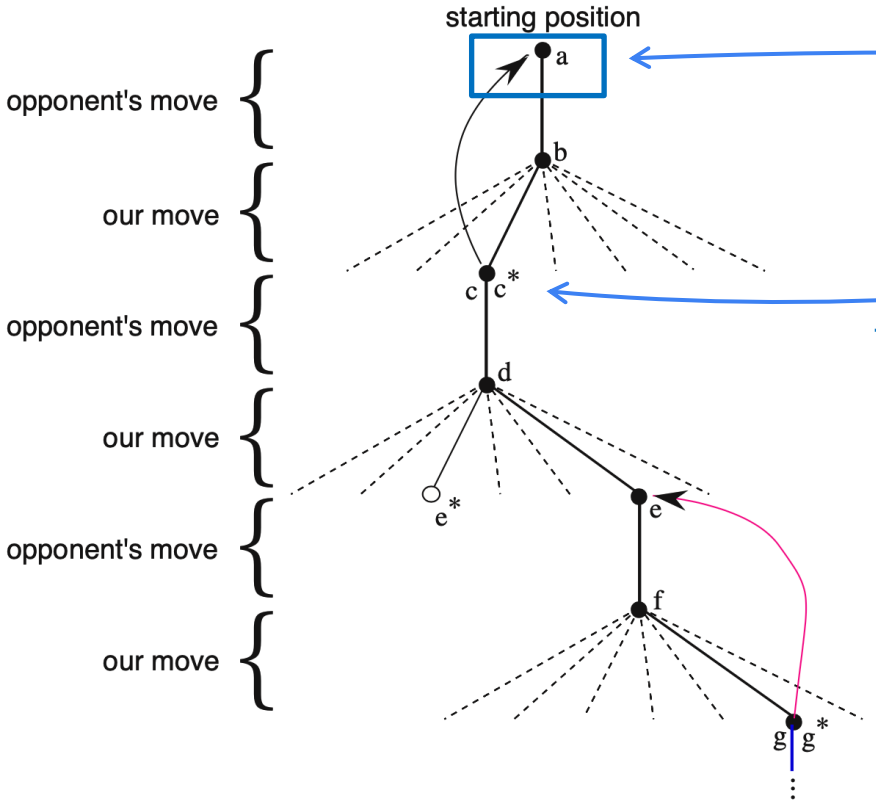
- $V(s_t) \leftarrow V(s_t) + \alpha[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$

The next state at timestep t+1

The current state at timestep t



Remember Tic-Tac-Toe?



The current state at timestep t

$$V(a) \leftarrow V(a) + \alpha[V(c) - V(a)]$$

The next state at timestep t+1

$$V(e) \leftarrow V(e) + \alpha[V(g) - V(e)]$$

Value update: $V(s_t) \leftarrow V(s_t) + \alpha[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$
where r_{t+1} can be 0, and γ can be 1 If the task is episodic with guaranteed termination.

Summary

- Dynamic programming: needs accurate knowledge of environment dynamics
- Monte Carlo: learn from complete sample returns
- Temporal Difference: Look at the difference between the current estimate of the value of a state and the discounted value of the next state and the reward received.